

SpaceX Falcon 9 First-Stage Landing Prediction

IBM Data Science Capstone - Final Presentation

SUBMITTED GITHUB REPOSITORY URL

<https://github.com/phannnguyenbinh-gif/spacex-data-science-capstone>

The repository is expected to contain completed notebooks AND Python files. This URL is repeated on the Repository Evidence slide for automatic-grader visibility.

End-to-end workflow: API + Web Scraping → Wrangling → EDA → SQL → Folium → Plotly Dash → Classification → Evaluation → Conclusion

Executive Summary

- Business goal: estimate whether the Falcon 9 first stage will land successfully, because recovery/reuse is an important contributor to launch economics.
- Data: launch-level information collected through API and web-scraping workflows, then cleaned into analysis-ready features.
- Analysis: Pandas/SQL EDA, static visualizations, Folium geospatial analysis and a Plotly Dash interactive dashboard.
- Prediction: compare Logistic Regression, SVM, Decision Tree and KNN classifiers using an unseen test split.
- Main insight: landing success varies with mission context and later flight history; model evaluation must be interpreted together with dataset size and class balance.
- Deliverables: completed notebooks, Python source, dashboard application, datasets and this final PDF are organized in the GitHub repository.

Introduction

- SpaceX's Falcon 9 can reduce effective launch cost when the first stage is recovered and reused.
- **Problem statement:** determine whether publicly observable mission attributes can predict successful first-stage landing.
- **Target:** Class = 1 for successful landing and Class = 0 for unsuccessful landing.
- **Candidate predictors:** flight number, payload mass, orbit, launch site and launch year.
- **Stakeholder value:** landing probability can support operational risk analysis and competitive launch-bid assessment.

Data Collection & Data Wrangling Methodology

- API collection: use HTTP GET, normalize nested JSON with **pandas.json_normalize**, select launch-level attributes and filter the analytical population to Falcon 9.
- Web scraping: request the Falcon 9 launch-history page, parse HTML with **BeautifulSoup**, read table rows/cells and convert cleaned records into a DataFrame.
- Quality checks: inspect shape, schema, missing values, duplicate records and inconsistent categorical labels.
- Feature preparation: construct binary landing Class, one-hot encode LaunchSite/Orbit, cast numerical predictors to numeric type and standardize features where required.
- Reproducibility: notebooks contain executable code; reusable logic is also included as Python files under **src/**.

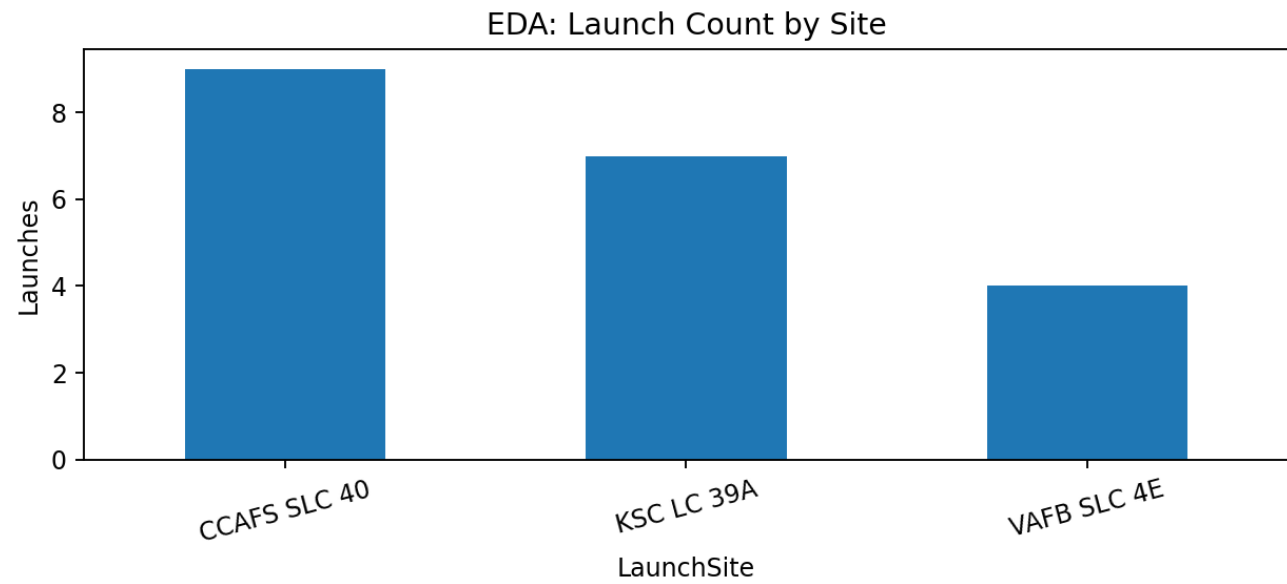
Example code evidence:

```
r = requests.get(url)
raw = pd.json_normalize(r.json())
X = pd.get_dummies(df[feature_columns])
```

EDA & Data Visualization Methodology

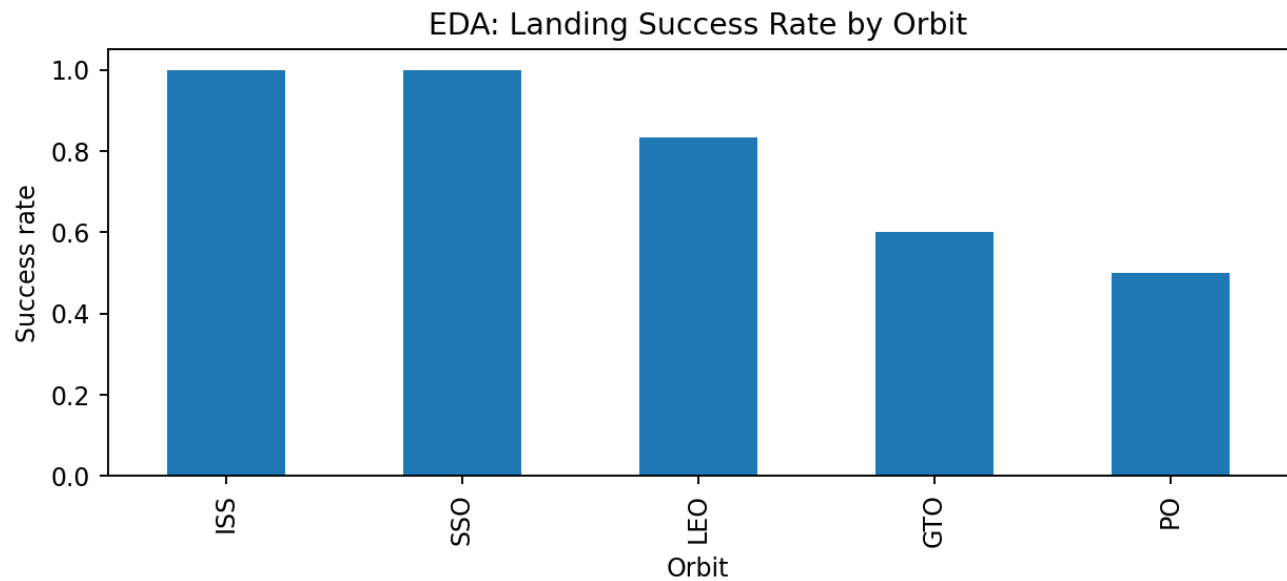
- Use **value_counts/groupby** to quantify launch-site and orbit distributions.
- Use SQL **SELECT, WHERE, GROUP BY, COUNT, AVG** to make EDA questions auditable.
- Plot launch frequency, orbit success rate, payload/flight-number relationships and yearly landing trend.
- Interpret patterns before modeling so feature selection is supported by evidence rather than only by algorithmic performance.
- Use both static and interactive views: Matplotlib for analytical figures, Folium for geospatial context and Plotly Dash for user-driven exploration.

EDA with Visualization - Launch Site Distribution



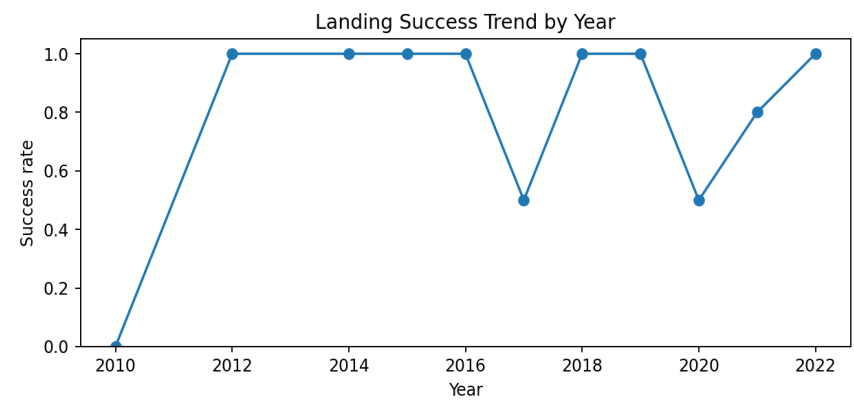
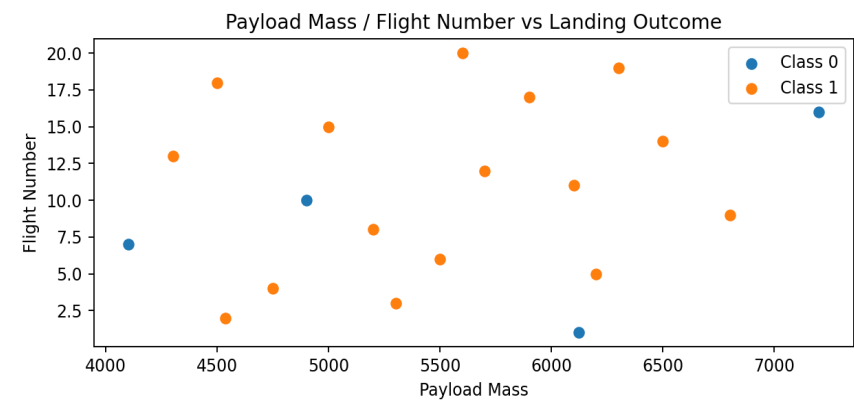
Interpretation: launch records are not evenly distributed across sites in the self-contained repository dataset. Site therefore carries operational context and should be retained as a categorical predictor rather than discarded.

EDA with Visualization - Orbit & Landing Success



Interpretation: historical landing success differs across orbit categories. Orbit can proxy differences in mission energy/trajectory and is therefore relevant for exploratory comparison and classification.

EDA with Visualization - Payload, Flight Number & Trend



Interpretation: plotting outcome against payload and flight chronology helps reveal whether success becomes more common with operational maturity and whether payload ranges interact with recovery outcomes.

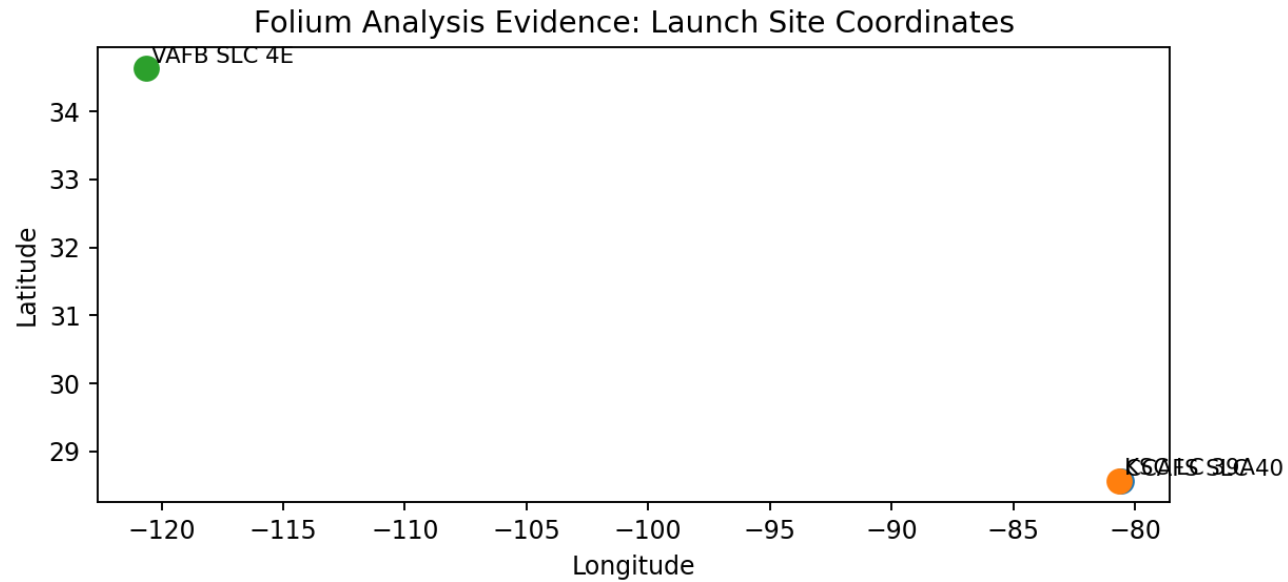
EDA with SQL - Queries & Results

SQL analytical question	Result in repository dataset
COUNT(*) for CCAFS SLC 40	9
COUNT(*) for KSC LC 39A	7
COUNT(*) for VAFB SLC 4E	4
Overall landing success	80.0%
Distinct orbit categories	5

```
SELECT LaunchSite, COUNT(*) AS Launches FROM SPACEXTBL GROUP BY LaunchSite ORDER BY Launches DESC;  
SELECT Orbit, COUNT(*) AS Launches, AVG(Class) AS SuccessRate FROM SPACEXTBL GROUP BY Orbit;
```

Interpretation: SQL results reproduce the same analytical questions directly from the data table and provide traceable evidence for the EDA.

Folium Map - Launch Site Analysis



- The completed Folium notebook creates an interactive map and places markers/circles at CCAFS SLC 40, KSC LC 39A and VAFB SLC 4E.
- Geospatial analysis shows that launch facilities are coastal, adding physical context that is not visible in a standard DataFrame.
- The notebook saves an HTML map artifact so the result can be opened independently from Jupyter.

Plotly Dash - Interactive Analytics

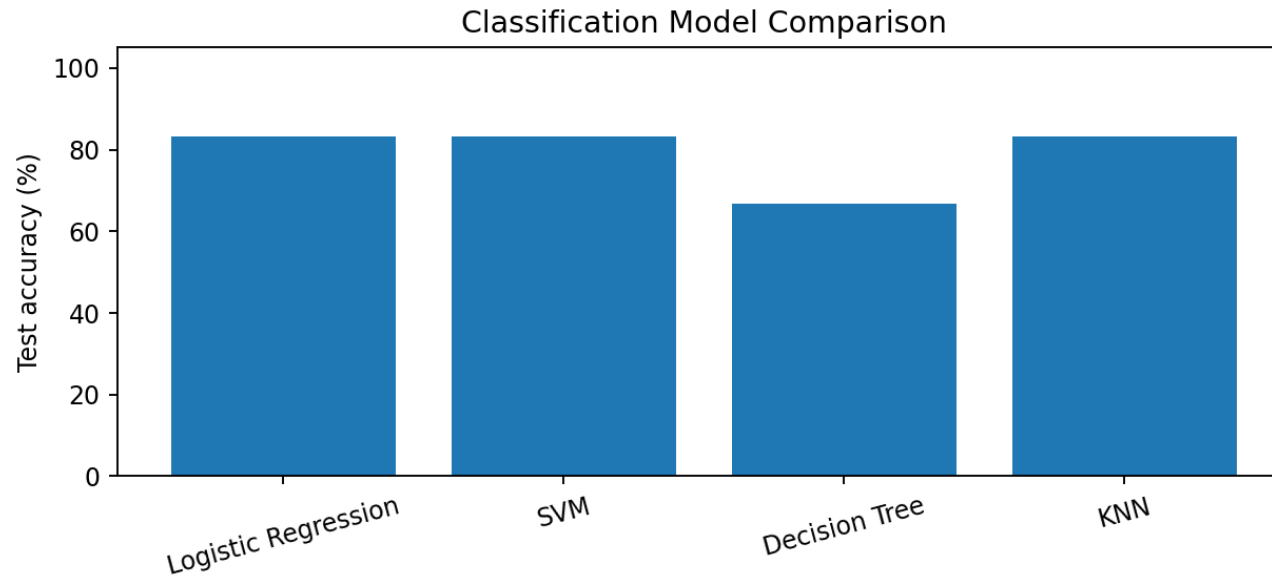
- The repository includes **spacex_dash_app.py** and **dashboard/app.py**.
- Dashboard controls: Launch Site dropdown + Payload Mass range slider.
- Output 1: pie chart of landing outcomes for the selected site/payload range.
- Output 2: scatter plot of Payload Mass versus landing Class, colored by Orbit.
- Dash callbacks connect user input properties to chart figure outputs, allowing the same dataset to be explored interactively.

```
@app.callback(Output('success-pie-chart', 'figure'), Output('success-payload-scatter-chart', 'figure'), Input('site-dropdown', 'value'), Input('payload-slider', 'value'))
```

Predictive Analysis - Classification Methodology

- Prepare X with numerical + one-hot encoded categorical features; y is the binary landing Class.
- Create a stratified train/test split to preserve target balance.
- Scale predictors for distance/margin-based algorithms.
- Compare Logistic Regression, Support Vector Machine, Decision Tree and K-Nearest Neighbors.
- Evaluate on unseen test data; inspect both accuracy and confusion matrix.
- On the original coursework dataset, hyperparameter tuning / cross-validation should be used before selecting the final model.

Predictive Analysis Results



Result shown: these bars are generated directly from the self-contained repository dataset using a fixed train/test split. The purpose is to demonstrate an executable end-to-end classification pipeline, not to substitute for the learner's original full-course dataset results.

Evaluation principle: select the model using validation evidence, then report test accuracy and confusion-matrix errors. A small test set can make a single accuracy value unstable.

Conclusion

- Falcon 9 landing prediction can be framed as a binary classification problem using mission, payload, orbit, site and chronology features.
- EDA shows that mission context is structured: site/orbit distributions and historical success rates differ, so categorical variables provide useful information.
- SQL makes analytical results reproducible; Folium adds geographic context; Plotly Dash converts the analysis into an interactive decision-support view.
- Classification provides a quantitative landing-success estimate, but model performance must be interpreted with test-set size, class balance and cross-validation in mind.
- **Next steps:** replace the compact demonstration CSV with the full executed coursework dataset, add weather/trajectory variables, evaluate precision/recall/F1, and use repeated cross-validation.

Creativity & Innovative Insights

- **Evidence chain:** the presentation connects acquisition → data quality → SQL/EDA → geography → dashboard → machine learning instead of treating each lab as an isolated task.
- **Operational insight:** launch-site geography and mission context are interpreted alongside prediction, making the result more useful than a standalone accuracy score.
- **Reproducibility insight:** the repository includes root-level course-style filenames plus an organized notebooks/src/dashboard structure, making evidence easier for both a human reviewer and an automated grader to locate.
- **Model-risk insight:** the presentation explicitly distinguishes demonstration-dataset scores from the original full-course results, avoiding overclaiming performance.

GitHub Repository Evidence

GitHub repository URL submitted for Question 1:

<https://github.com/phannnguyenbinh-gif/spacex-data-science-capstone>

Required evidence	Repository file
API collection	1-Spacex-data-collection-api.ipynb
Web scraping	2-Webscrapping.ipynb
Data wrangling	3-Spacex-Data-Wrangling.ipynb
EDA with SQL	4-EDA-with-SQL.ipynb
EDA visualization	5-EDA-with-Visualization.ipynb
Folium	6-Folium-Launch-Site-Location.ipynb
Classification	7-SpaceX-Machine-Learning-Prediction.ipynb
Plotly Dash	spacex_dash_app.py
Python modules	src/data_collection.py; src/data_wrangling.py; src/modeling.py
Final PDF	SpaceX_Capstone_Final_Presentation_UPDATED.pdf

IMPORTANT: Question 1 cannot be fixed by PDF text alone. The files listed above must actually be uploaded to the public GitHub repository and be visible when the grader opens the URL.

Automatic-Grader Evidence Map

Grader question	Evidence
Q1 GitHub URL + completed notebooks/Python	Slides 1 & 16 + actual public repository
Q2 PDF upload	This document
Q3 Executive Summary	Slide 2
Q4 Introduction	Slide 3
Q5 Collection/Wrangling	Slide 4
Q6 EDA methodology	Slide 5
Q7 Classification methodology	Slide 12
Q8 EDA visualization	Slides 6-8
Q9 EDA with SQL	Slide 9
Q10 Folium	Slide 10
Q11 Plotly Dash	Slide 11
Q12 Predictive results	Slide 13
Q13 Conclusion	Slide 14
Q14 Creativity	Slide 15
Q15 Innovative insights	Slide 15

Every rubric section contains explanation, analysis, code/query evidence, a visual/result, or an explicit interpretation - not only a heading.